

Name _____ Period _____

Skill 18.1 Exercise 1	
For each of the following, indicate whether the string operation is using a function or a method	
<pre>pet = "fido" pet_len = len(pet)</pre>	function
<pre>letters = "oliver" sorted_letters = sorted(letters)</pre>	function
<pre>fav_class = "Python" fav_class_lower = fav_class.lower()</pre>	method
<pre>msg = "hello world" msg.replace("h", "j")</pre>	method
Summarize the difference between a function and a method.	
String functions are called with a string as an argument whereas methods are called at the end of the string and have their own arguments	

Skill 18.2 Exercise 1
You need to write a program that generates usernames for students at Timberline. The usernames are in all lower case and will use the following convention: first_initial + last_name. For example, Lady Gaga would have the following username, lgaga. Write a function called <i>make_un</i> that could be used to create a username. Your function should accept two parameters (one for the first name and one for the last name) and return the username.
<pre>def make_un(f_name, l_name): first_let = f_name[0, 1] un = (first_let + l_name).lower()</pre>

Skill 18.3 Exercise 1
A single line in a CSV file looks as follows, Alice, Seattle, WA 98108 Use the split command to store the data as a list, then store each value in its respective field: name, city, state, zip (notice there is a space between the state abbreviation and the zip code!)
<pre>info = "Alice, Seattle, WA 98108" data = info.split(",") state_zip = data[2].split() name = data[0] city = data[1] state = state_zip[0] zip = state_zip[1] print(name, city, state, zip)</pre>

Name _____ Period _____

Skill 18.3 Exercise 2

Below is the full text for William Carlos Williams poem *Spring Storm*.

```
spring_storm_text = \
"""The sky has given over
its bitterness.
Out of the dark change
all day long
rain falls and falls
as if it would never end.
Still the snow keeps
its hold on the ground.
But water, water
from a thousand runnels!
It collects swiftly,
dappled with black
cuts a way for itself
through green ice in the gutters.
Drop after drop it falls
from the withered grass-stems
of the overhanging embankment."""
```

Use `split()` to break up the text into individual lines

```
text_list = spring_storm_text.split("\n")
```

What is the average length of each line?

```
total = 0
for line in text:
    total += len(line)

avg = total/len(text)
print(avg)
```

Name _____ Period _____

Skill 18.4 Exercise 1

You have a list of numeric values representing a data record, and you want to format it like a CSV line – that is a line separated by commas.

```
fields = ["California", "39500000", "84000", "45000"]
```

Use *join* to build a CSV row

```
fields_text = ",".join(fields)
```

Many datasets — especially large ones — use **TSV instead of CSV** because tabs are less likely to appear naturally in text fields. In the space below, use *join* to create a string of tab separated values.

```
fields_text = "\t".join(fields)
```

Skill 18.5 Exercise 1

Below is a line of raw data,

```
raw_line = " California , large population , tech hubs "
```

Use the *strip* and *join* methods to create cleaned list like the one below,

California | large population | tech hubs

```
raw_line = " California , large population , tech hubs "  
raw_list = raw_line.split(",")  
# cleaned_list = [v.strip() for v in raw_list]  
cleaned_list = []  
for entry in raw_list:  
    cleaned_list.append(entry.strip())  
cleaned_line = "|".join(cleaned_list)  
print(cleaned_line)
```

Name _____ Period _____

Skill 18.6 Exercise 1

You import a dataset where the income is stored in the following format,

```
raw_income = ["$45,000", "$82,300", "$9,500"]
```

Before you can use this data you must remove the dollar signs and commas, and convert the strings to numbers.

Clean the data in the list above, store the cleaned data in a new list called *cleaned income*.

```
clean_income = []  
for value in raw_income:  
    temp = value.replace("$", "")  
    temp2 = temp.replace(",", "")  
    clean_income.append(int(temp2))
```

Skill 18.7 Exercise 1

You receive text data from a product review, and you want to check whether the review mentions a keyword (e.g., “price”) and where it appears in the text.

```
review = "The battery life is great, but the price is too high."
```

```
print(review.find("price"))
```

Skill 18.7 Exercise 2

You receive logs from a data pipeline that *should* look like CSV, but the values are mixed with extra text,

```
log_line = [INFO] user_id=42, load_time=351ms, status=OK
```

You want to extract the load_time value 351 as an integer. Indicate how you could do this using the string methods you have learned about so far.

```
log_line = "[INFO] user_id=42, load_time=351ms, status=OK"  
load_time = log_line.find("load_time=")  
time_end = log_line.find(",", load_time)  
time_start = load_time + len("load_time=")  
time_ms = log_line[time_start:time_end]  
time = int(time_ms.strip("ms"))  
print(time)
```

Name _____ Period _____

Skill 18.8 Exercise 1

Write a function called *state_summary* that takes,

- a state name
- its population
- its median income

as arguments and returns a nicely formatted summary string using *format()*.

The output should look as follows,

State: California
Population: 39500000
Median Income: \$84,000

```
def state_summary(state, population, income):  
    result = "State: {state}\n"  
    result += "Income: ${income}\n"  
    result += "Population: {population}"  
    return result.format(state = state, income = income, population = population)
```